

Removing Noisy Instances and Weak Attributes using hybrid Decision Tree and Naïve Bayes

Ikramul Hasan Moral

Department of Computer Science and Engineering
United International University
Dhaka, Bangladesh
imoral223489@bscse.uiu.ac.bd

Dewan Md. Farid

Professor, Department of Computer Science and Engineering
United International University
Dhaka, Bangladesh
dewanfarid@cse.uiu.ac.bd

Abstract—A Naïve Bayes (NB) filter removes misclassified training instances before a decision tree (DT) is trained. A tree selects and weights attributes for NB. This study combines those steps in both orders and compares their accuracy with the individual algorithms and the base classifiers. Cleaning is evaluated under two protocols: once on the full dataset before cross-validation, and separately within each training fold. The comparison uses C4.5 and NB on ten datasets, records accuracy and macro-F1, and tracks the training instances removed from each class. The fold-specific protocol estimates performance without using test cases to clean the training data.

Index Terms—decision tree, Naïve Bayes, hybrid classifier, noise filtering, attribute weighting, cross-validation

I. INTRODUCTION

Decision trees and Naïve Bayes (NB) handle classification in different ways. A tree can split on details that do not hold in new data. NB assumes that attributes act independently within each class. Attributes that add little information can weaken it. Farid and colleagues described two algorithms for these problems in 2014 [1]. One uses NB to remove training instances before growing a tree. The other uses a tree to choose and weight attributes for NB.

The two steps can be applied to the same training set. Their order matters because the first step changes the data seen by the second. If NB removes instances first, the tree chooses attributes from fewer rows. If the tree chooses attributes first, NB judges instances using fewer attributes. Either order can end in a tree or an NB classifier.

How we test the methods also matters. A cleaner fitted on the full dataset uses information from cases that later appear in the test folds. Instance removal is especially direct: cases rejected by the cleaner can disappear from the test set itself [2]. We compare that procedure with one in which every cleaning step is fitted again inside each training fold. The second procedure keeps test rows out of the cleaning decision.

The study compares the two orders with the original single-step algorithms and with uncleaned NB and tree baselines. Accuracy and macro-F1 measure classification. We also record the number and class of removed instances, because a higher score alone does not show what the cleaner discarded.

II. PROBLEM SETTING

An instance that NB misclassifies is not necessarily noisy [3]. Its label may be correct even if the model has trouble predicting it. The filter still removes that instance. We record how many rows it deletes and which classes they belong to. Deletion can change the balance of the training set.

The attribute step makes a different decision. It builds a tree and keeps the attributes tested by that tree. An attribute used near the root gets a higher weight than one first used deeper in the tree. An attribute never tested is dropped. This rule is tied to the fitted tree. If the rows change, the tree and its choice of attributes may change too.

An instances-first path asks which attributes the tree selects after NB has removed rows. An attributes-first path asks which rows NB removes after the tree has selected attributes. Both orders use the same evaluation settings and base classifiers.

III. THE THREE ALGORITHMS

A. Algorithm 1: NB removes noisy instances

NB is trained on the training set and then classifies every training instance. Each instance it gets wrong is removed, and a C4.5 tree is grown on those that remain. The judge is fitted only on the training data supplied to this step. If deletion would leave fewer than two classes, the original training rows are kept so the final classifier can still be fitted.

B. Algorithm 2: a tree weights attributes for NB

A C4.5 tree is grown on the training set. An attribute receives a larger weight when the tree tests it closer to the root. An attribute the tree never tests receives zero weight and is removed. NB uses the remaining weights when it predicts a class. Selection is fitted again whenever the training rows change; weights from a different fold are not reused.

C. Algorithm 3: the combined hybrid

Algorithm 3 applies both steps to the same training set (Algorithm 1). With instances first, plain NB removes the instances it misclassifies; the tree then chooses and weights

attributes on the remaining rows. With attributes first, the tree chooses and weights the attributes before weighted NB judges the instances. In that path, the weights from the tree are passed to the NB judge as well as the final NB classifier. The final classifier in either path is weighted NB or a C4.5 tree.

The cleaner returns the selected attributes and the training rows that remain after filtering. The final classifier is fitted on those rows. A test fold is prepared with the same selected attributes, but none of its rows are removed. Cleaning can change the training sample, but every held-out case still has to be scored.

Algorithm 1 Combined hybrid (Algorithm 3)

Require: training set D ; order $\in \{\text{instances-first, attributes-first}\}$; final classifier $\in \{\text{NB, DT}\}$

Ensure: a trained classifier M and the chosen attributes S

- 1: $S \leftarrow$ all attributes; $W_j \leftarrow 1$ for all j
 - 2: **if** order = attributes-first **then**
 - 3: grow a tree on D ; set S and W as in Algorithm 2
 - 4: keep only the attributes in S
 - 5: remove from D every instance that NB with weights W misclassifies
 - 6: **else**
 - 7: remove from D every instance that plain NB misclassifies
 - 8: grow a tree on D ; set S and W as in Algorithm 2
 - 9: keep only the attributes in S
 - 10: **end if**
 - 11: **if** final classifier = NB **then**
 - 12: train NB on D with weights W
 - 13: **else**
 - 14: grow a C4.5 tree on D
 - 15: **end if**
 - 16: **return** M, S
-

IV. EXPERIMENTAL METHOD

A. Datasets and tools

The evaluation uses the ten datasets of the 2014 study (Table I). Nine come from the UCI repository and one is NSL-KDD [4]. C4.5 is Weka’s J48 [5] with its default settings. NB follows the textbook model described in the original paper: nominal attributes use counts with add-one smoothing, and numeric attributes use a normal distribution per class. All nominal attributes stay as they are, so that a tree splits each one into one branch per value, as C4.5 does.

The original attribute columns are used for this comparison. Keeping each nominal attribute as one column matters because the tree chooses and weights attributes, not individual encoded values. The base classifiers and cleaning paths start with the same columns. Their inputs differ only after a cleaning step removes rows or columns.

TABLE I
DATASETS, AS READ FROM OUR COPIES OF THE FILES

Dataset	Instances	Attributes	Classes
Breast cancer	286	9	2
Contact lenses	24	4	3
Diabetes	768	8	2
Glass	214	9	6 ^a
Iris	150	4	3
Soybean	683	35	19
Vote	435	16	2
Image segmentation	2310 ^b	19	7
Tic-tac-toe	958	9	2
NSL-KDD (20% train)	25192	41	22 ^a

^aOne listed class has no instances in the available file. ^bBoth available UCI files are combined.

B. Evaluation protocols

The main evaluation uses ten repetitions of stratified 10-fold cross-validation, with a different random split for each repetition. Within each split, all methods begin with the same training and test rows. The cleaner and final classifier are fitted on the training rows. The test rows are never filtered; they only lose columns that the training-fold cleaner removed. This is the fold-specific protocol used to estimate performance on unseen cases.

A second protocol fits the cleaning steps once on the full dataset, then cross-validates only the final classifier on the cleaned data. The filter in this protocol can use labels from rows that later appear in a test fold. It can also remove those rows before they are scored. We report its accuracy only to show the effect of cleaning before the split. It does not estimate performance on unseen cases. Because filtering changes the sample, its folds are not treated as paired with the folds of the main evaluation.

The difference between settings matters most for methods that delete instances. When a method only removes attributes, every row remains in the test set, although the selected attributes may still reflect information from the full dataset. We examine the two effects separately.

C. Methods compared

The uncleaned C4.5 and NB classifiers are the baselines for each final classifier. Algorithm 1 pairs instance removal with a tree, while Algorithm 2 pairs tree-selected attributes with NB. Algorithm 3 has four versions: two orders of cleaning, each ending in either NB or a tree.

Two further comparisons help explain any difference. Algorithm 1 ending in NB shows what instance removal does without switching to a tree. Algorithm 2 ending in a tree shows what attribute selection does without switching to NB. All methods use the same evaluation rules. We can then tell whether a score changed because of cleaning or because the final classifier changed.

D. Measures and statistical test

Accuracy is the share of test instances classified correctly. Macro-F1 averages the F1 score of each class, so a large class cannot alone determine the summary. Both measures are reported for each dataset before any average across datasets is considered. The per-dataset scores show whether an average hides a large loss on one dataset.

For methods that remove instances, the analysis records how many training rows are removed from each class. A removal rate is compared with that class’s share of the original training fold. These records help distinguish a small, even reduction from a filter that mostly removes one class. The number of attributes removed by the tree is recorded separately. Test-row counts stay unchanged in the fold-specific protocol.

Average accuracy over the repetitions is calculated for each dataset and method. Paired method comparisons then use those ten dataset-level averages in a Wilcoxon signed-rank test [6] at the 0.05 level. We report the score difference alongside the test result. If the test is not significant, we report no clear difference; ten datasets may miss a small one.

V. RESULTS

TABLE II
FOLD-SPECIFIC CLASSIFICATION RESULTS (PERCENT)

Method	Accuracy	Macro-F1
NB	–	–
C4.5	–	–
Algorithm 1	–	–
Algorithm 2	–	–
Instances first, NB	–	–
Instances first, DT	–	–
Attributes first, NB	–	–
Attributes first, DT	–	–

VI. LIMITATIONS

The ten datasets allow a comparison with the original study, but they do not represent every kind of classification problem. The available files also differ from some class and row counts printed in that study, as noted in Table I. We use the available files and report those differences.

The NB judge marks a row for removal when its own prediction is wrong. That does not prove the row’s label is incorrect. A correctly labelled but hard case can be removed, and the method has no second judge to check it. The removal counts help interpret any accuracy change. They cannot tell which removed labels were truly noisy.

The full-data-clean protocol shows how much the measured score changes when cleaning occurs before cross-validation. It cannot establish which protocol the 2014 study used. The fold-specific estimate remains the main result for comparing methods on held-out data.

VII. CONCLUSION

The hybrid applies NB instance filtering and tree-based attribute selection in either order. The single-step and uncleaned methods show what each step adds. Fold-specific testing scores every held-out row, and the removal counts show which classes lose training rows. The measured results are needed to tell whether either order improves classification.

REFERENCES

- [1] D. M. Farid, L. Zhang, C. M. Rahman, M. A. Hossain, and R. Strachan, "Hybrid decision tree and naïve Bayes classifiers for multi-class classification tasks," *Expert Systems with Applications*, vol. 41, no. 4, pp. 1937–1946, 2014.
- [2] G. C. Cawley and N. L. C. Talbot, "On over-fitting in model selection and subsequent selection bias in performance evaluation," *Journal of Machine Learning Research*, vol. 11, no. 70, pp. 2079–2107, 2010.
- [3] C. E. Brodley and M. A. Friedl, "Identifying mislabeled training data," *Journal of Artificial Intelligence Research*, vol. 11, pp. 131–167, 1999.
- [4] M. Tavallae, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*. IEEE, 2009, pp. 1–6.
- [5] I. H. Witten, E. Frank, M. A. Hall, and C. J. Pal, *Data Mining: Practical Machine Learning Tools and Techniques*, 4th ed. Cambridge, MA: Morgan Kaufmann, 2016.
- [6] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, no. 1, pp. 1–30, 2006.